

Week 2 · 실습 워크북: 참조를 잊고, state를 분석하고, 역순으로 지웁니다

📌 참고

이 문서는 실습 파트(43분)입니다. 개념은 같은 `lecture/` 폴더의 `개념워크북` 을 먼저 읽어보세요.

표시 읽는 법 - **[실습]** 직접 입력합니다 - **[확인]** 화면에 나온 것과 문서를 눈으로 대조합니다 - **[함정]** 미리 알고 피해야 할 실수입니다 - **[관찰 🕒]** 빈칸을 채웁니다 (제출물이 됩니다) - **(심화 ⭐)** 먼저 끝낸 분만 하세요. 안 해도 오늘 목표는 달성입니다

오늘 끝나면 이걸 할 수 있습니다. VPC부터 EC2까지 7개를 배포하고, `tfstate` 안에서 내가 적지 않은 의존성 그래프를 찾아내고, 역순으로 전부 지웁니다.

! 꼭 기억

AWS 단어가 낯설면 개념워크북 0번을 먼저 보세요. VPC, 서브넷, IGW, 라우트 테이블, SG가 각각 무엇인지 비유로 설명해 뒀습니다. 2분이면 읽습니다. 이를 읽지 않고 이 문서를 따라오면 왜 이것을 만드는지 모른 채 입력만 하게 됩니다.

오늘 만들 것

인터넷에 연결된 리눅스 서버 한 대. 그러려면 AWS 리소스가 이만큼 필요합니다.

VPC + 퍼블릭 서브넷 + IGW + 라우트 테이블 + 연결(association) + 시큐리티 그룹 + EC2 t3.micro. 리소스 7개, 그 사이를 잇는 참조 11개.

📌 참고

오늘부터는 세션이 끝나면 반드시 `destroy` 를 합니다. Week1의 빈 버킷과 달리 EC2는 켜져 있는 동안 요금이 발생합니다. 액수와 계산은 Block C 첫머리에서 다시 설명합니다. 지금은 "끝에 지우는 단계가 있다"만 기억하시면 됩니다.

그리고 NAT Gateway와 Elastic IP는 만들지 마세요. 이 둘은 프리티어가 없고 안 쓰고 있어도 시간당 과금됩니다.

! 꼭 기억

작업 폴더는 이 리포의 `practice/` 하나뿐입니다. 새 폴더를 만들지 마세요. `versions.tf`, `providers.tf`, `variables.tf` 는 이미 준비돼 있고, 여러분은 `network.tf`, `compute.tf`, `outputs.tf` 의 `# TODO` 만 채우면 됩니다.

파일	무슨 일을 하나	오늘 수정하나
<code>versions.tf</code>	Terraform-provider 버전 고정	읽기만
<code>providers.tf</code>	리전 + 모든 리소스에 자동으로 붙는 태그	읽기만

파일	무슨 일을 하나	오늘 수정하나
<code>variables.tf</code>	입력 변수 선언 + 값 검사	읽기만
<code>network.tf</code>	VPC·서브넷·IGW·라우트 테이블·연결	채웁니다 (TODO 4개)
<code>compute.tf</code>	AMI 조회·시큐리티 그룹·EC2	채웁니다 (TODO 2개)
<code>outputs.tf</code>	출력값 6개	채웁니다 (TODO 2개)

오늘의 의존성 지도

Week1에서는 명령의 흐름(Scope → Author → Initialize → Plan → Apply)을 따라갔습니다. 오늘은 리소스 사이의 의존 관계를 따라갑니다.

이 의존성 지도를 오늘 세 번 사용합니다. Block A에서 잇고, Block B에서 apply 로그와 대조하고, Block C에서 뒤집습니다.

0층 · 먼저 AWS에서 조회하는 것 (plan 때 이미 결과를 받아둡니다)

`data.aws_ami.al2023`

`data.aws_availability_zones.available`

1층 · 아무도 안 기다리는 유일한 리소스

`aws_vpc.main`

2층 · VPC만 있으면 되는 셋 → 동시에 시작

`aws_subnet.public`

`aws_internet_gateway.gw``aws_security_group.web`

← ① vpc

← ③ vpc

← ⑧ vpc

← ② AZ데이터

3층 · 2층이 끝나야 시작 → 둘이 동시에

`aws_route_table.public``aws_instance.web`

← ④ vpc

← ⑨ subnet

← ⑤ igw

← ⑩ sg

← ⑪ ami데이터

4층 · 맨 마지막

`aws_route_table_association.public`

← ⑥ subnet

← ⑦ route_table

참조 번호와 뜻은 개념워크북 1번 표와 같습니다.

참고

← 는 "이 값을 기다린다"는 뜻입니다. 그래서 나중에 `terraform graph` 를 돌리면 `aws_subnet.public -> aws_vpc.main` (서브넷이 VPC를 기다린다)로 똑같은 방향이 나옵니다. 지도와 graph의 방향을 통일해 뒀으니 헷갈릴 일이 없습니다.

그리고 위에서 아래로 내려갈수록 나중에 만들어집니다. 0층이 가장 먼저, 4층이 가장 마지막입니다.

스텝	여기서 잇는 참조	하는 일
A-3	① ②	서브넷 작성

스텝	여기서 있는 참조	하는 일
A-4	③ ④ ⑤ ⑥ ⑦	IGW · 라우트 테이블 · 연결 작성
A-5	-	plan. <code>Plan: 5 to add</code>
A-6	-	네트워크 5개 apply
B-1	⑧ ⑨ ⑩ ⑪	SG · EC2 작성
B-2	-	plan. <code>Plan: 2 to add</code>
B-3	-	컴퓨터 2개 apply
C-1 · C-2	전부 역방향	7개 destroy (리프 노드부터)

참고

길을 잃었을 때는 "지금 몇 번 참조를 잇는 중이지?"만 확인하면 됩니다. 오늘 배울 것을 한 문장으로 줄이면, 참조를 다 이으면 순서는 적지 않아도 된다는 것입니다.

팁

4층이 EC2가 아닙니다. 인자가 참조 두 줄뿐인 `route_table_association` 이 가장 마지막입니다. 왜 그런지는 B-3에서 로그로 확인합니다.

오늘 나올 숫자를 미리 정리해 둡니다. 이것과 다르면 문제가 있는 것입니다.

지점	나와야 하는 숫자
A-5 plan (네트워크만)	<code>Plan: 5 to add, 0 to change, 0 to destroy.</code>
A-6 후 <code>state list</code>	7줄 (리소스 5 + 데이터 소스 2)
B-2 plan (EC2 추가)	<code>Plan: 2 to add, 0 to change, 0 to destroy.</code>
B-4 <code>state list</code>	9줄 (리소스 7 + 데이터 소스 2)
C-1 destroy plan	<code>Plan: 0 to add, 0 to change, 7 to destroy.</code>
C-2 완료	<code>Destroy complete! Resources: 7 destroyed.</code>

참고

7 이 세 군데 나오는데 뜻이 다 다릅니다. 헷갈리지 않게 미리 구분해 둡니다. - 오늘 만드는 리소스 7개 (데이터 소스 제외)
- A-6 후 `state list` 의 7줄 (그때까지 만든 리소스 5개 + 데이터 소스 2개) - destroy되는 7개 (첫 번째와 같은 7)

Block A. 워크스루 (12분) : 강사와 함께

참조 ① ~ ⑦ 을 읽고, 네트워크 5개만 먼저 apply

참고

네트워크만 먼저 만드는 이유는, EC2까지 한 번에 만들면 apply 로그가 길어져서 "같은 순간에 시작한 것들"을 눈으로 확인하기 어렵기 때문입니다. 오늘의 관찰 지점을 유지하기 위해 두 번에 나눠서 만듭니다.

[실습] A-1. 지난주와 달라진 폴더: 파일이 왜 쪼개졌나

```
cd practice
ls
terraform version          # 1.9.0 이상인지 확인
aws sts get-caller-identity # AWS 자격증명이 유효한지 확인
jq --version               # B-4와 C-0에서 씁니다. 없으면 아래 TIP 참고
```

팁

jq 가 없다고 나오면 지금 설치해두세요. state 파일(JSON)에서 원하는 부분만 골라 보여주는 도구입니다. macOS는 `brew install jq`, Ubuntu/WSL은 `sudo apt install -y jq`. 설치가 안 되어도 실습은 진행할 수 있지만 B-4에서 결과 확인이 불편해집니다.

Week1은 `main.tf` 하나였는데 오늘은 `network.tf` 와 `compute.tf` 로 나뉘어 있습니다. 왜일까요?

Terraform은 파일 경계를 구분하지 않습니다. 현재 폴더의 `.tf` 파일을 전부 읽어서 하나로 합칩니다(Week1 A-2). 그래서 파일을 쪼개는 기준은 문법이 아니라 사람이 찾기 편한 단위입니다.

즉 `network.tf` 에 있는 `aws_vpc.main` 을 `compute.tf` 에서 바로 참조할 수 있습니다. 파일이 달라도 Terraform에게는 같은 코드입니다.

주의

터미널은 하나만 쓰세요. 두 개에서 동시에 명령을 돌리면 `Error acquiring the state lock` 에러가 납니다. Terraform이 state를 두 명이 동시에 고치는 것을 막기 때문입니다.

[실습] A-2. 내 값 채우기: `project_name` · `my_ip`

```
cp example.tfvars terraform.tfvars
curl -4 ifconfig.me          # 내 공인 IPv4 주소 한 줄이 나옵니다
```

📌 참고

`terraform.tfvars` 는 변수의 실제 값을 적어두는 파일입니다. Terraform이 자동으로 읽어옵니다. 사람마다 값이 다르고(내 IP가 들어가니까) 비밀이라서 커밋하지 않습니다(`.gitignore` 에 있습니다).

`terraform.tfvars` 를 열어서 두 값을 바꿉니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w2-{본인-github-id}" # 예: boaz26-w2-kdh1834
my_ip       = "1.2.3.4"                    # curl -4 결과. /32 를 붙이지 마세요
instance_type = "t3.micro"
```

! 꼭 기억

`my_ip` 에 `/32` 를 붙이지 마세요. 코드 쪽에서 `"${var.my_ip}/32"` 로 자동 조립합니다. `1.2.3.4/32` 를 넣으면 `1.2.3.4/32/32` 가 되어버립니다.

실수하면 `variables.tf` 가 값을 검사해 차단합니다. 값이 `CHANGE-ME` 거나 형식이 틀리면 plan이 아예 시작되지 않습니다.

```
Error: Invalid value for variable
  my_ip는 1.2.3.4 처럼 순수 IPv4여야 합니다. /32나 CIDR을 넣지 마세요.
```

이 실패는 미리 발견되는 편이 낫습니다. 이 검사가 없으면 SG를 만드는 순간(= apply 도중)에

`InvalidParameterValue` 로 실패합니다. 그때는 VPC, 서브넷, IGW가 이미 만들어져 있어서 복구가 가장 번거로운 시점입니다.

💡 팁

`curl` 에 `-4` 를 꼭 붙이세요. 안 붙이면 IPv6 주소(`2001:...`)가 나올 수 있는데, 그건 `/32` 를 붙일 수 없는 형식입니다.

! 꼭 기억

CP1. Discord에 `project_name` 한 줄만 올려주세요. apply 전에 오타를 확인합니다.

🚫 절대 금지

`my_ip` 값은 Discord에 올리지 마세요. 본인 공인 IP입니다. `curl -4 ifconfig.me` 가 IPv4 한 줄을 출력했는지 여부만(`OK` / `실패`) 보고하세요.

[실습] A-3. 첫 참조 ① ②: 서브넷 만들기

지금 하는 일은 VPC 안에 서버가 배치될 서브넷을 하나 만드는 것입니다.

network.tf 의 첫 TODO입니다. data "aws_availability_zones" 와 aws_vpc.main 은 이미 채워져 있습니다.

```
resource "aws_subnet" "public" {
  vpc_id            = aws_vpc.main.id                # 참조 ①
  cidr_block        = "10.0.1.0/24"
  availability_zone = data.aws_availability_zones.available.names[0] # 참조 ②

  map_public_ip_on_launch = true

  tags = { Name = "${var.project_name}-public" }
}
```

한 줄씩 읽어봅시다.

줄	무슨 뜻인가
vpc_id = aws_vpc.main.id	"이 서브넷은 저 VPC 안에 있습니다". 참조 ①
cidr_block = "10.0.1.0/24"	이 서브넷이 쓸 주소 범위. 10.0.1.무엇 256개 (개념워크북 0-3)
availability_zone = ...names[0]	AWS에서 조회한 AZ 목록 중 첫 번째. 참조 ②
map_public_ip_on_launch = true	이 서브넷에서 뜨는 서버에 퍼블릭 IP를 자동으로 붙여준다
tags = { Name = ... }	콘솔에서 식별할 이름

읽는 규칙은 하나입니다. 왼쪽은 인자 이름(provider가 정함), 오른쪽은 참조(내가 씀).

```
vpc_id = aws_vpc.main.id
  ^      ^
  |      +---- 오른쪽: 내가 붙인 주소(aws_vpc.main) + 속성(.id)
  +----- 왼쪽: aws_subnet 이 요구하는 인자 이름 (못 바꿈)
```

여기서 세 가지를 확인합니다.

(1) AZ를 하드코딩하지 않습니다. "ap-northeast-2a" 라고 직접 적는 대신 AWS에서 조회합니다. AZ 이름이 계정마다 다른 데이터센터에 붙기 때문이고, AZ마다 쓸 수 있는 인스턴스 타입도 다르기 때문입니다. 자세한 이유는 개념워크북 7번에 있습니다.

(2) map_public_ip_on_launch = true 한 줄이 퍼블릭 IP가 붙는 원인입니다. 이 서브넷에서 뜨는 서버는 퍼블릭 IPv4를 자동으로 받습니다. 그래서 우리는 Elastic IP를 만들지 않습니다. EIP는 안 쓰고 있어도 시간당 과금됩니다.

(3) 방금 개념워크북 0-5의 세 조건 중 첫 번째를 만들었습니다. 나머지 둘이 어느 스텝인지만 확인하고 넘어가세요.

① 퍼블릭 IP가 있는가	: map_public_ip_on_launch = true	<- A-3 (방금 완료)
② 인터넷 경로가 있는가	: IGW + 라우트 테이블 0.0.0.0/0 + 연결	<- A-4 (다음)
③ 방화벽이 허용하는가	: SG의 egress 전체 허용	<- B-1

[실습] A-4. 참조 ③ ④ ⑤ ⑥ ⑦: IGW · 라우트 테이블 · 연결

지금 하는 일은 인터넷 게이트웨이(IGW)를 만들고, 라우트 테이블에 인터넷으로 나가는 경로를 적고, 그 테이블을 아까 만든 서브넷에 연결하는 것입니다.

`network.tf` 의 남은 TODO 3개를 채웁니다.

```
resource "aws_internet_gateway" "gw" {
  vpc_id = aws_vpc.main.id # 참조 ③
  tags   = { Name = "${var.project_name}-igw" }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id # 참조 ④

  route {
    cidr_block = "0.0.0.0/0"           # "어디로 가든" (= 인터넷 전체)
    gateway_id = aws_internet_gateway.gw.id # 이 IGW로 내보낸다 (참조 ⑤)
  }

  tags = { Name = "${var.project_name}-rt-public" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id      # 어느 서브넷에 (참조 ⑥)
  route_table_id = aws_route_table.public.id # 어느 라우트 테이블을 (참조 ⑦)
}
```

그리고 `outputs.tf` 의 첫 TODO도 지금 채웁니다.

```
output "vpc_id" {
  description = "생성된 VPC ID"
  value       = aws_vpc.main.id
}

output "subnet_id" {
  description = "퍼블릭 서브넷 ID"
  value       = aws_subnet.public.id
}

output "subnet_az" {
  description = "서브넷이 놓인 AZ. data.aws_availability_zones 가 골라준 값"
  value       = aws_subnet.public.availability_zone
}
```

여기서 처음 보는 것이 두 개 있습니다.

(1) `route` 는 리소스가 아니라 인자입니다.

`resource "aws_route_table"` 안에 종괄호로 들어간 `route { }` 는 별개의 리소스가 아니라 라우트 테이블이 가진 속성입니다. 겉모습이 `resource` 블록과 비슷해서 헷갈리기 쉬운데, 앞에 `resource` 라는 단어가 없으면 리소스가 아닙니다. 이렇게 블록 안에 들어가는 것을 인라인 블록이라고 부릅니다.

(2) `association` 은 연결하는 행위 자체가 리소스입니다.

AWS 콘솔에는 이걸 위한 독립 화면이 없습니다(라우트 테이블 화면의 탭으로 보일 뿐입니다). 그런데 Terraform에서는 독립된 리소스 하나이고, 인자가 참조 두 줄뿐입니다. 그래서 의존성 지도에서 가장 마지막 층에 놓입니다. 서브넷과 라우트 테이블이 둘 다 만들어져야 비로소 만들 수 있으니까요.

[실습] A-5. `fmt` → `init` → `validate` → `plan`

```
terraform fmt      # 들여쓰기 정렬. 고친 파일 이름만 출력됩니다
terraform init     # provider 다운로드 (예습에서 이미 했으면 몇 초)
terraform validate  # 문법·정합성 검사
terraform plan
```

참고

순서에 이유가 있습니다. `validate` 는 provider가 있어야 검사할 수 있어서 `init` 다음에 실행합니다. `fmt` 는 아무 때나 해도 됩니다.

[확인] `plan` 맨 위: 데이터 소스가 먼저 읽힌다

```
data.aws_ami.al2023: Reading...
data.aws_availability_zones.available: Reading...
data.aws_availability_zones.available: Read complete after 1s [id=ap-northeast-2]
data.aws_ami.al2023: Read complete after 1s [id=ami-04bb4ddfc0e51bb5e]
```

데이터 소스 2개가 `plan` 단계에서 이미 읽혔습니다. `compute.tf` 의 SG와 EC2는 아직 주석 상태인데도 `data "aws_ami"` 블록은 주석 처리되어 있지 않으므로 읽힙니다. Week1 A-8에서 본 그 `Reading...` 줄이 오늘은 두 개입니다.

[확인] 서브넷 블록: 확정된 값과 아직 모르는 값

```
# aws_subnet.public will be created
+ resource "aws_subnet" "public" {
  + availability_zone           = "ap-northeast-2a"
  + availability_zone_id       = (known after apply)
  + cidr_block                  = "10.0.1.0/24"
  + id                         = (known after apply)
  + map_public_ip_on_launch    = true
  + tags                       = {
    + "Name" = "boaz26-w2-kdh1834-public"
  }
  + tags_all                   = {
    + "ManagedBy" = "terraform"
    + "Name"       = "boaz26-w2-kdh1834-public"
    + "Project"    = "boaz26-w2-kdh1834"
    + "Study"      = "boaz-terraform-26"
    + "Week"       = "2"
  }
  + vpc_id                    = (known after apply)
}
```

세 줄만 나란히 비교해 보세요.

줄	값	왜 이런가
<code>cidr_block</code>	<code>"10.0.1.0/24"</code>	내가 직접 적은 값이라 plan 때 이미 압니다
<code>availability_zone</code>	<code>"ap-northeast-2a"</code>	데이터 소스가 읽어온 값이라 plan 때 이미 압니다
<code>vpc_id</code>	<code>(known after apply)</code>	VPC를 만들어야 정해지는 값 → 참조 ①의 결과

`(known after apply)` 가 참조의 증거입니다. "아직 값을 모른다"가 곧 "그것보다 나중에 만들어야 한다"입니다.

그리고 `tags` 와 `tags_all` 을 비교해보세요. `tags` 는 내가 이 리소스에 직접 쓴 것(`Name` 1개)이고, `tags_all` 은 `providers.tf` 의 `default_tags` 와 합쳐진 최종 결과(5개)입니다. 내가 이 리소스에 쓰지 않은 태그 4개(`Project`, `Study`, `Week`, `ManagedBy`)가 자동으로 붙었습니다. `providers.tf` 를 열어보면 왜인지 보입니다.

```
Plan: 5 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + subnet_az = "ap-northeast-2a"
  + subnet_id = (known after apply)
  + vpc_id    = (known after apply)
```

! 꼭 기억

`Plan: 5 to add` 가 아니면 멈추세요. - 4가 나왔다면 TODO가 하나 덜 채워졌습니다 - 7이 나왔다면 `compute.tf` 의 TODO를 미리 채운 것입니다. 그것도 틀린 것은 아니지만 A-6의 로그 관찰이 어려워집니다

[함정] plan이 멈추거나 이상할 때

증상	원인	해결
<code>my_ip</code> 는 1.2.3.4 처럼... 에러	<code>terraform.tfvars</code> 가 <code>CHANGE-ME</code> 그대로	A-2를 다시
<code>var.project_name</code> 을 입력하라며 멈춤	<code>terraform.tfvars</code> 를 안 만들	Ctrl+C로 빠져나오고 파일부터 만드세요. 여기서 임의의 값을 입력하면 그 값으로 apply됩니다
<code>No changes.</code>	TODO가 아직 주석 상태	주석을 풀고 채우기
<code>Reference to undeclared resource</code>	주소 오타 (<code>aws_vpc.main</code> ↔ <code>aws_vpc.mian</code>)	<code>validate</code> 가 줄 번호를 알려줍니다
<code>Inconsistent dependency lock file</code>	<code>init</code> 을 안 함	<code>terraform init</code>
<code>Your query returned no results</code>	AMI 필터에 일치하는 이미지가 없음	리전이 <code>ap-northeast-2</code> 인지 확인

[실습] A-6. 네트워크만 먼저 apply: 순서를 로그로 보기

```
terraform apply    # 물어보면 yes
```

[확인] apply 로그를 읽는 법: 같은 순간에 뜬 것이 같은 층

```
aws_vpc.main: Creating...
aws_vpc.main: Creation complete after 2s [id=vpc-0f9e8d7c6b5a4321f]
aws_subnet.public: Creating...
aws_internet_gateway.gw: Creating...
aws_subnet.public: Creation complete after 1s [id=subnet-0a1b2c3d4e5f67890]
aws_internet_gateway.gw: Creation complete after 1s [id=igw-0123456789abcdef0]
aws_route_table.public: Creating...
aws_route_table.public: Creation complete after 1s [id=rtb-0fedcba9876543210]
aws_route_table_association.public: Creating...
aws_route_table_association.public: Creation complete after 0s [id=rtbassoc-0aaa...]
```

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

```
subnet_az = "ap-northeast-2a"
subnet_id = "subnet-0a1b2c3d4e5f67890"
vpc_id    = "vpc-0f9e8d7c6b5a4321f"
```

읽을 곳이 세 군데입니다.

(1) **aws_vpc.main** 이 가장 먼저 단독으로 실행됩니다. 아무것도 참조하지 않는 유일한 리소스라서 여기가 출발점입니다. 이것이 끝나기 전에는 다른 리소스를 시작할 수 없습니다.

(2) **subnet** 과 **igw** 의 **Creating...** 이 연달아 표시됩니다. 둘은 서로를 참조하지 않으므로 동시에 시작합니다(2층). 개념워크북 4번 표에는 2층이 3개라고 되어 있는데, 지금은 SG를 아직 안 만들었으니 2개만 보이는 게 맞습니다.

(3) **association** 이 마지막입니다. 서브넷(⑥)과 라우트 테이블(⑦)이 둘 다 있어야 하니까요.

그런데 저는 어디에도 순서를 적지 않았습니다. 참조 ①~⑦을 이어놓았을 뿐입니다.

```
terraform state list
```

```
data.aws_ami.al2023
data.aws_availability_zones.available
aws_internet_gateway.gw
aws_route_table.public
aws_route_table_association.public
aws_subnet.public
aws_vpc.main
```

7줄입니다. 리소스 5 + 데이터 소스 2.

데이터 소스는 만든 것이 없는데도 state에 기록됩니다. 다음 plan에서 "그때 값과 지금 값이 같은가"를 비교해야 하기 때문입니다.

[관찰 🍷] A-6 기록

- `Creating...` 이 같은 순간에 뜬 리소스 이름 전부:
- `aws_route_table.public` 이 `aws_internet_gateway.gw` 보다 늦게 시작한 이유를 참조 번호로:
- 가장 마지막에 `Creation complete` 가 뜬 리소스: / 의존성 지도의 예측과 같은가:
- `state list` 줄 수: / 그중 실제 AWS 리소스는 몇 개:
- `subnet_az` 값: (옆 사람과 같은가? 다르면 왜 다를까?)

[실습] A-7. terraform graph : 그래프를 텍스트로 보기 (선택)

지금 하는 일은 Terraform이 내부적으로 만든 그래프를 텍스트로 출력해서 확인하는 것입니다.

```
terraform graph | grep '\->' | sed 's/"//g; s/;/;/g' | sort
```

```
aws_internet_gateway.gw -> aws_vpc.main
aws_route_table_association.public -> aws_route_table.public
aws_route_table_association.public -> aws_subnet.public
aws_route_table.public -> aws_internet_gateway.gw
aws_subnet.public -> aws_vpc.main
aws_subnet.public -> data.aws_availability_zones.available
```

A → B 는 "A가 B를 기다린다"는 뜻입니다. 의존성 지도의 방향을 뒤집은 것이고, 이쪽이 실제 코드에 적힌 참조 방향입니다.

📌 참고

왜 `aws_route_table.public -> aws_vpc.main` (참조 ④)이 안 보이냐면, `terraform graph` 의 기본 출력은 요약본이라서 다른 경로로 이미 보장되는 간선은 생략하기 때문입니다. 라우트 테이블은 IGW를 기다리고, IGW는 VPC를 기다립니다. 그러니 "라우트 테이블 → VPC"는 자동으로 보장되죠.

코드에는 ④가 분명히 있지만 이 출력에는 나오지 않습니다. 그리고 B-4에서 볼 state의 `dependencies` 배열에는 다시 나타납니다.

💡 팁

`dot` (Graphviz)을 설치하지 마세요. 세션 시간이 부족합니다. 그림으로 꼭 보고 싶으면 `terraform graph` 출력을 통째로 복사해서 <https://dreampuf.github.io/GraphvizOnline/> 에 붙여넣으면 됩니다.

Block B. 각자 진행 (20분) : EC2 · state 분석 · drift

참조 ⑧ ⑨ ⑩ ⑪ 을 읽고, state를 분석합니다

! 꼭 기억

B-1 ~ B-4가 필수입니다. B-5부터는 심화입니다.

강사가 "8분 남았습니다"라고 공지하면 어디에 있는 Block C로 이동하세요. 못 끝낸 분도 destroy는 반드시 합니다. 절반만 만들어진 것도 과금됩니다.

[실습] B-1. 참조 ⑧ ⑨ ⑩ ⑪: 시큐리티 그룹 + EC2

지금 하는 일은 서버 앞에 붙일 시큐리티 그룹(SG)을 만들고, 드디어 EC2 인스턴스를 띄우는 것입니다.

`compute.tf` 의 TODO 2개를 채웁니다. `data "aws_ami"` 는 이미 채워져 있습니다.

```
resource "aws_security_group" "web" {
  name          = "${var.project_name}-web-sg"
  description   = "boaz w2 lab: SSH from my IP only"
  vpc_id        = aws_vpc.main.id # 참조 ⑧

  ingress {
    description = "SSH from my IP"          # 들어오는 것
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["${var.my_ip}/32"]       # 내 IP 하나만
  }

  egress {
    description = "all outbound"            # 나가는 것
    from_port   = 0
    to_port     = 0
    protocol    = "-1"                      # -1 = 모든 프로토콜
    cidr_blocks = ["0.0.0.0/0"]             # 어디로든 나갈 수 있음
  }

  tags = { Name = "${var.project_name}-web-sg" }
}

resource "aws_instance" "web" {
  ami              = data.aws_ami.al2023.id # 참조 ⑪
  instance_type    = var.instance_type
  subnet_id        = aws_subnet.public.id   # 참조 ⑨
  vpc_security_group_ids = [aws_security_group.web.id] # 참조 ⑩

  root_block_device {
    volume_type = "gp3"          # 서버의 디스크
    volume_size = 8               # 8 GiB
    delete_on_termination = true # 서버 지울 때 디스크도 같이 삭제
  }

  metadata_options {
```

```

    http_tokens = "required"          # 보안 설정. 오늘은 몰라도 됩니다 (아래 NOTE)
  }

  tags = { Name = "${var.project_name}-web" }
}

```

📌 참고

`root_block_device` 와 `metadata_options` 는 오늘 몰라도 됩니다. 앞의 것은 디스크 설정(8GiB짜리 SSD를 붙이고, 서버를 지울 때 같이 지운다), 뒤의 것은 켜두면 좋은 보안 기본값입니다. 실무에서 매번 쓰는 것들이라 미리 넣어뒀을 뿐, 오늘의 학습 목표(참조·그래프·state)와는 상관없습니다. 궁금하면 개념워크북 0-4의 EBS 항목을 보세요.

📌 참고

시큐리티 그룹은 양방향 방화벽입니다. `ingress` (인그레스)는 들어오는 트래픽 규칙이고, 여기서는 "내 IP에서 오는 22번 포트(SSH)만 허용"입니다. `egress` (이그레스)는 나가는 트래픽 규칙이고, 여기서는 "어디로든 다 나갈 수 있음"입니다.

나가는 걸 열어두는 이유는 서버가 패키지를 받거나 업데이트할 수 있어야 하기 때문입니다. 개념워크북 0-5의 ③번 조건 (방화벽이 허용하는가)이 이 `egress` 입니다.

`outputs.tf` 의 두 번째 TODO도 채웁니다.

```

output "ami_id" {
  description = "data.aws_ami 가 찾아낸 AL2023 AMI ID"
  value       = data.aws_ami.al2023.id
}
output "instance_id" {
  description = "EC2 인스턴스 ID (destroy 전에 기록해 두면 증빙이 됩니다)"
  value       = aws_instance.web.id
}
output "instance_public_ip" {
  description = "자동 할당된 퍼블릭 IPv4. EIP가 아니라서 stop/start 하면 바뀝니다. Discord에 올리지 말 것"
  value       = aws_instance.web.public_ip
}

```

여기서 확인할 것은 다섯 가지입니다.

(1) SG에 `vpc_id` 를 빼먹으면 안 됩니다. 빼면 SG가 기본 VPC에 만들어지고, EC2 생성 단계에서 이렇게 실패합니다.

```

Error: creating EC2 Instance: InvalidParameterValue:
  Security group sg-xxx and subnet subnet-yyy belong to different networks.

```

SG와 EC2가 서로 다른 VPC에 있다는 뜻입니다.

여기서 말하는 기본 VPC는 AWS 계정을 만들면 리전마다 미리 만들어져 있는 연습용 VPC입니다. `vpc_id` 를 안 적으면 SG가 거기에 만들어지고, 우리가 만든 VPC와는 다른 네트워크가 되어버립니다.

(2) `vpc_security_group_ids` 는 리스트입니다. SG는 서버 한 대에 여러 개 붙일 수 있어서, 하나만 붙여도 `[ ]` 가 필요합니다.

- (3) `ami = data.aws_ami.al2023.id` 가 오늘 유일한 `data.` 참조입니다. 리소스는 `aws_subnet.public` 처럼 바로 시작하지만, 데이터 소스는 앞에 `data.` 가 붙습니다.
- (4) `cidr_blocks = ["0.0.0.0/0"]` 로 22번을 열지 마세요. 몇 분 안에 전 세계 스캔 트래픽이 붙습니다. 반드시 내 IP /32 만.
- (5) `associate_public_ip_address` 를 쓰지 마세요. 퍼블릭 IP는 서브넷의 `map_public_ip_on_launch` 한 곳에서만 통제합니다. 두 군데에 적으면 나중에 어느 쪽이 실제로 적용되는지 헷갈리고, 이 값을 바꾸면 서버가 통째로 재생성(-/+)됩니다.

🚫 절대 금지

키페어를 만들지 마세요. 오늘은 SSH 접속을 하지 않습니다.

코드로 키를 만들면 비밀키가 tfstate에 평문으로 저장되고, `.pem` 파일이 실수로 커밋되는 사고가 발생할 수 있습니다.

그럼 SG에 22번 규칙은 왜 남기냐고요? ① 문법을 배우기 위해 ② B-5 drift 실습에서 사용하기 위해 ③ "필요한 만큼만 여는" 습관을 들이기 위해서입니다.

오늘의 성공 기준은 접속이 아니라 `instance_state = running` + 퍼블릭 IP 할당입니다.

[확인] B-2. 두 번째 plan: (known after apply) 가 줄었습니다

```
terraform fmt && terraform validate
terraform plan
```

```
# aws_instance.web will be created
+ resource "aws_instance" "web" {
  + ami                  = "ami-04bb4ddfc0e51bb5e"
  + instance_type       = "t3.micro"
  + private_ip          = (known after apply)
  + public_ip           = (known after apply)
  + subnet_id           = "subnet-0a1b2c3d4e5f67890"
  + vpc_security_group_ids = (known after apply)
  ...
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:

```
+ ami_id                = "ami-04bb4ddfc0e51bb5e"
+ instance_id           = (known after apply)
+ instance_public_ip    = (known after apply)
```

A-5와 비교해보세요. `subnet_id` 가 (known after apply) 가 아니라 확정된 값입니다.

서브넷은 A-6에서 이미 만들어졌고 state에 값이 적혀 있으니 Terraform이 그 값을 압니다. 참조 ⑨는 그대로 있지만, 가리키는 대상이 이미 존재하는 것이죠.

반면 `vpc_security_group_ids` 는 아직 (known after apply) 입니다. SG를 이번에 만들 거니까요(참조 ⑩).

💡 팁

`ami_id` 값을 옆 사람과 비교해보세요. 같은 리전, 같은 필터라면 같은 값이 나옵니다. 데이터 소스는 같은 조건에 같은 결과를 반환하니까요.

다만 `most_recent = true` 라서 다음 주에 돌리면 다른 값이 나올 수 있습니다. 그리고 `ami` 는 `-/+` (교체) 속성입니다. 코드를 안 고쳤는데 EC2가 재생성될 수 있다는 뜻입니다(B-6에서 확인).

[실습] B-3. apply: 서버 생성

```
terraform apply # yes
```

⚠️ 주의

`Still creating... [10s elapsed]` 가 정상입니다. EC2는 provider가 `running` 상태가 될 때까지 기다리므로 30~60초 걸립니다. 화면이 멈춘 것처럼 보여도 Ctrl+C를 절대 누르지 마세요.

Ctrl+C로 강제 종료하면 이런 일이 생깁니다. 1. state 잠금이 남아서 다음 명령이 `Error acquiring the state lock` 으로 실패합니다 2. state에는 안 적혔는데 AWS에는 존재하는 리소스가 생길 수 있습니다

2번이 실제로 심각한 문제입니다. Terraform이 모르는 리소스라서 `destroy` 로도 삭제되지 않고 요금만 계속 청구됩니다.

[확인] apply 성공 신호

```
aws_security_group.web: Creating...
aws_security_group.web: Creation complete after 3s [id=sg-0abc123def456789a]
aws_instance.web: Creating...
aws_instance.web: Still creating... [10s elapsed]
aws_instance.web: Still creating... [20s elapsed]
aws_instance.web: Still creating... [30s elapsed]
aws_instance.web: Creation complete after 33s [id=i-0123456789abcdef0]
```

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

```
ami_id           = "ami-04bb4ddfc0e51bb5e"
instance_id      = "i-0123456789abcdef0"
instance_public_ip = "3.35.xxx.xxx"
subnet_az        = "ap-northeast-2a"
subnet_id        = "subnet-0a1b2c3d4e5f67890"
vpc_id           = "vpc-0f9e8d7c6b5a4321f"
```

SG가 먼저, EC2가 나중입니다. 참조 ⑩ 때문입니다. 그리고 EC2 하나가 나머지 6개를 다 합친 것보다 오래 걸립니다.

! 꼭 기억

CP2. 아래 두 가지를 `project_name` 과 함께 Discord에 텍스트로 올려주세요. (스크린샷 말고 텍스트) - `Apply complete! Resources: 2 added.` 줄 - `terraform state list` 출력 전체

🚫 절대 금지

`instance_public_ip` 값은 올리지 마세요. 22번 포트가 열려 있는 서버의 주소입니다. 화면에서만 확인하고, Discord에는 `state list` 만 올리세요.

[관찰 📝] B-3 기록

- `Creation complete` 순서: `___` → `___`
- EC2 하나 만드는 데 걸린 초: `___` / A-6의 네트워크 5개 전체는: `___`
- 의존성 지도의 4층은 `association`인데, 로그에서 가장 늦게 끝난 것은 무엇이었나: `___`
- 왜 다른가 (한 문장): `___`
- 퍼블릭 IP가 붙었는데 EIP는 안 만들었습니다. 어느 인자 덕분인가: `___`

[실습] B-4. state 분석 ⭐

apply는 리소스를 만드는 것으로 끝나지 않습니다. state에 기록하는 것까지 포함합니다. 그 state를 분석합니다.

📌 참고

아래에 `jq` 라는 명령이 나옵니다. JSON 파일에서 원하는 부분만 골라 보여주는 도구입니다. 없으면 `brew install jq` (macOS) / `sudo apt install jq` (Ubuntu)로 설치하세요. 없어도 `terraform state show` 로 대부분 볼 수 있습니다.

① 주소 목록 보기

```
terraform state list
```

```
data.aws_ami.al2023
data.aws_availability_zones.available
aws_instance.web
aws_internet_gateway.gw
aws_route_table.public
aws_route_table_association.public
aws_security_group.web
aws_subnet.public
aws_vpc.main
```

9줄 = 리소스 7 + 데이터 소스 2. `Plan: 5 to add` + `Plan: 2 to add` = 7과 딱 맞습니다.

② 하나만 자세히 확인하기

```
terraform state show aws_instance.web
```

이 중에서 확인할 것: `id` · `private_ip` · `public_ip` · `subnet_id` · `ami` · `instance_state` · `tags` vs `tags_all`.

```
terraform state show aws_subnet.public | grep -E "availability_zone|cidr_block|map_public"
```

`availability_zone = "ap-northeast-2a"`. 데이터 소스가 선택한 값이 여기 저장돼 있습니다.

③ 파일을 직접 열어보기: 최상위 키 7개

```
ls -la terraform.tfstate
jq 'keys' terraform.tfstate
jq '{version, terraform_version, serial, lineage}' terraform.tfstate
```

```
{
  "version": 4,
  "terraform_version": "1.15.8",
  "serial": 3,
  "lineage": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

키	뜻
<code>version: 4</code>	state 파일의 포맷 버전. Terraform 버전이 아닙니다
<code>serial</code>	state가 저장된 횟수. 저장할 때마다 늘어납니다
<code>lineage</code>	이 state의 고유 식별 번호. 다른 인프라의 state를 잘못 덮어쓰는 사고를 방지합니다

! 꼭 기억

`serial` 숫자는 사람마다 다르게 나옵니다. apply 횟수가 아니라 state를 저장한 횟수이기 때문입니다. apply 말고도 output이 바뀌거나 refresh 결과가 달라지면 저장이 한 번 더 일어납니다. 그래서 apply를 두 번 했는데 `serial` 이 2가 아닐 수 있습니다.

이 번호는 Week3에서 활용됩니다. 같은 state를 두 사람이 각자 고쳤을 때 "내가 가진 것이 이전 버전이다"를 판단하는 근거가 이 숫자입니다.

④ 리소스 목록과 속성

```
jq -r '.resources[] | "\(.mode) \(.type).\(.name)"' terraform.tfstate
jq '.resources[] | select(.type=="aws_instance") | .instances[0].attributes
  | {id, ami, instance_type, subnet_id, private_ip, public_ip, tags, tags_all}' terraform.tfstate
```

🚫 절대 금지

`attributes`에는 우리가 코드에 쓴 것보다 훨씬 많은 값이 들어 있습니다. AWS가 자동으로 정한 기본값까지 전부입니다. 그리고 `plan`이 `~` 인지 `-/+` 인지 계산하는 근거가 바로 이 값들입니다. 전부 평문입니다.

⑤ ★ `dependencies`: 내가 적지 않은 그래프

```
jq -r '.resources[] | "\(.type).\(.name) <- \(.instances[0].dependencies // [] | join(", "))"' terraform.tfstate
```

```
aws_instance.web <- aws_security_group.web, aws_subnet.public, aws_vpc.main, data.aws_ami.al2023, ...
aws_internet_gateway.gw <- aws_vpc.main
aws_route_table.public <- aws_internet_gateway.gw, aws_vpc.main
aws_route_table_association.public <- aws_route_table.public, aws_subnet.public, aws_vpc.main, ...
aws_security_group.web <- aws_vpc.main
aws_subnet.public <- aws_vpc.main, data.aws_availability_zones.available
aws_vpc.main <-
```

이어서 아래 명령의 출력과 나란히 비교해 보세요.

```
terraform graph | grep '\->' | sed 's/"//g; s/;/;/g' | sort
```

! 꼭 기억

왼쪽은 `.tf` 파일에서 계산한 그래프, 오른쪽은 state에 저장된 그래프입니다. 같습니다.

그런데 왜 state에 또 적어둘까요? `.tf` 파일을 다 지워버려도 `terraform destroy`가 올바른 순서로 동작해야 하기 때문입니다. 코드가 사라지면 참조도 사라지지만, state의 `dependencies`는 남습니다. C-2에서 삭제 순서가 지켜지는 근거가 바로 이것입니다.

저는 이 배열을 적은 적이 없습니다. 참조 ①~⑪에서 자동으로 추출되어 여기 적었습니다.

📌 참고

두 가지를 더 설명합니다.

(1) 목록이 참조 개수보다 많아 보입니다. `dependencies`에는 직접 의존 외에 간접 의존까지 들어갑니다. association의 목록에 `aws_vpc.main`이 보이는 이유입니다(association → 서브넷 → VPC로 이어지니까요).

(2) `aws_instance.web`의 목록에 `aws_route_table_association.public`이 없습니다. EC2는 인터넷 경로가 연결되기를 기다리지 않습니다. 참조가 없으니까요.

오늘은 문제가 안 되지만, 서버 부팅 스크립트로 패키지를 받는다면 인터넷이 아직 없어서 실패할 수 있습니다. 그때 필요한 것이 `depends_on`입니다(개념워크북 6번).

⑥ 기계가 읽는 JSON으로 출력하기

```
terraform show -json | jq '.values.root_module.resources[] | {address, depends_on}'
```

```
terraform show -json > state.json # 만들었으면 git status를 반드시 확인!
```

🚫 절대 금지

`state.json` 은 state와 같은 수준의 민감 정보입니다. `.gitignore` 로 제외되어 있지만 직접 확인하세요.

⑦ state pull

```
terraform state pull | jq -c '{version, serial, lineage, n:(.resources|length)}'  
jq .serial terraform.tfstate # 값이 그대로인지 확인
```

지금처럼 state가 내 노트북에 있을 때는 `cat terraform.tfstate` 와 거의 같습니다. 이 명령이 실제로 의미가 있는 것은 Week3의 S3 원격 저장소입니다. 짝인 `state push` 는 위험해서 오늘은 쓰지 않습니다.

[관찰 🍷] B-4 기록 (이 문항들이 오늘 실습의 핵심입니다)

- `state list` 총 줄 수: / 그중 `data.` 로 시작하는 줄:
- `state show aws_instance.web` 의 `private_ip` : / `instance_state` :
- `tags` 의 키 개수: / `tags_all` 의 키 개수: / 차이가 나는 이유:
- `aws_instance.web` 의 `dependencies` 배열 전체:
- 그 배열이 의존성 지도의 몇 번에 해당하나:
- 그 배열에 `aws_route_table_association.public` 이 없는 이유:
- `aws_vpc.main` 의 `dependencies` : (왜 그럴까?)
- 최상위 `serial` 값: (apply는 두 번 했는데 왜 이 숫자일까? 힌트는 바로 위 IMPORTANT 박스)
- `mode` 가 "data" 인 항목 수:
- `terraform_version` :
- `state.json` 이 `git status` 에 보이는가: (보이면 즉시 Discord)

[실습] B-5. drift 만들어보기: 콘솔에서 직접 수정해 보기

지금 하는 일은 Week1의 [함정] 콘솔에서 직접 수정하면 을 직접 재현하는 것입니다.

AWS 콘솔 → VPC → Your VPCs → 내 VPC 선택 → Tags 탭 → Manage tags → Name 값 뒤에 `-BYHAND` 추가 → Save.

💡 팁

콘솔 조작이 어려우면 CLI로도 할 수 있습니다.

```
# {본인-project_name} 을 terraform.tfvars 에 적은 값으로 바꿔서 실행하세요  
aws ec2 create-tags --region ap-northeast-2 \
```

```
--resources "${terraform output -raw vpc_id)" \  
--tags Key=Name,Value={본인-project_name}-vpc-BYHAND
```

이제 두 명령의 차이를 봅시다.

```
terraform plan -refresh-only      # ① AWS가 바뀐 것만 보여줌. state를 건드리지 않음
```

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply" which may have affected this plan:

```
# aws_vpc.main has been changed  
~ resource "aws_vpc" "main" {  
    id      = "vpc-0f9e8d7c6b5a4321f"  
    ~ tags   = {  
        ~ "Name" = "boaz26-w2-kdh1834-vpc" -> "boaz26-w2-kdh1834-vpc-BYHAND"  
    }  
}
```

```
terraform plan                    # ② Terraform이 무엇을 하겠다고 하는가
```

```
# aws_vpc.main will be updated in-place  
~ resource "aws_vpc" "main" {  
    id      = "vpc-0f9e8d7c6b5a4321f"  
    ~ tags   = {  
        ~ "Name" = "boaz26-w2-kdh1834-vpc-BYHAND" -> "boaz26-w2-kdh1834-vpc"  
    }  
    ~ tags_all = {  
        ~ "Name" = "boaz26-w2-kdh1834-vpc-BYHAND" -> "boaz26-w2-kdh1834-vpc"  
        # (4 unchanged elements hidden)  
    }  
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

! 꼭 기억

-> 방향을 보세요. 콘솔에서 바꾼 값 -> 코드에 적힌 값 입니다.

Terraform은 콘솔 변경을 반영하는 게 아니라 되돌립니다.

코드가 정답이고 AWS는 그 사본입니다. 이게 IaC입니다. 그래서 IaC를 도입한 팀은 콘솔에서 직접 수정하는 것을 금지합니다. 다음 apply에서 경고 없이 원래 값으로 되돌아가기 때문입니다.

```
terraform apply      # yes ← 복구가 별도 작업이 아닙니다. apply 자체가 복구입니다
```

 참고

Terraform이 감지하지 못하는 drift도 있습니다. ① 코드에 적지 않은 속성이 바뀐 경우 ② 콘솔에서 새로 만든 리소스 (`import` 하기 전에는 Terraform이 인식하지 못합니다)

그래서 state에 없으면 Terraform에게는 존재하지 않는 것이 됩니다.

[관찰 🏹] B-5 기록

- `plan -refresh-only` 의 맨 위 Note 한 줄을 그대로: `___`
- `terraform plan` 의 숫자: `___ to add, ___ to change, ___ to destroy`
- Terraform이 제안한 방향 (콘솔 값 유지 / 코드 값으로 복구): `___`
- `tags` 와 `tags_all` 이 둘 다 바뀐 이유: `___`

[실습] B-6 (심화 ⭐). ~ vs -/+ 체험: plan만, apply 절대 금지

⚠ 주의

이 스텝에서는 절대 `apply` 를 누르지 마세요. plan만 보고 즉시 원래대로 되돌립니다. apply하면 EC2가 삭제되고 새로 만들어집니다(시간 낭비 + 과금).

대조군: `instance_type` 은 ~ (제자리 수정)

`terraform.tfvars` 에서 `instance_type = "t2.micro"` 로 바꾸고 plan.

```
# aws_instance.web will be updated in-place
~ resource "aws_instance" "web" {
    id          = "i-0123456789abcdef0"
    ~ instance_type = "t3.micro" -> "t2.micro"
}
```

Plan: 0 to add, 1 to change, 0 to destroy.

EC2에는 인스턴스 타입을 바꾸는 API가 있어서 중지 → 변경 → 시작으로 처리됩니다. 인스턴스 ID가 유지되므로 ~ 입니다.

하지만 서버는 잠깐 멈춥니다. ~ 가 곧 "무해"는 아닙니다.

본 실습: 서브넷 `cidr_block` 은 -/+, 그리고 연쇄 교체

`network.tf` 에서 `cidr_block = "10.0.1.0/24"` → `"10.0.2.0/24"` (숫자 한 글자) 로 바꾸고 plan.

Resource actions are indicated with the following symbols:

-/+ destroy and then create replacement

```
# aws_instance.web must be replaced
-/+ resource "aws_instance" "web" {
    ~ subnet_id = "subnet-0a1b2c3d4e5f67890" -> (known after apply) # forces replacement
}
```

```
# aws_route_table_association.public must be replaced
-/+ resource "aws_route_table_association" "public" {
    ~ subnet_id = "subnet-0a1b2c3d4e5f67890" -> (known after apply) # forces replacement
}

# aws_subnet.public must be replaced
-/+ resource "aws_subnet" "public" {
    ~ cidr_block = "10.0.1.0/24" -> "10.0.2.0/24" # forces replacement
}

Plan: 3 to add, 0 to change, 3 to destroy.
```

세 가지를 읽으세요.

- (1) **# forces replacement** 주석의 위치. 리소스 헤더가 아니라 원인이 된 속성 줄의 맨 끝입니다. 그 속성이 교체의 원인입니다.
- (2) **연쇄 교체**. 한 줄 고쳤는데 3개가 재생성됩니다. 서브넷이 삭제되면 그 **id**를 참조하던 association과 EC2도 같이 삭제됩니다. 의존성 지도가 곧 교체가 전파되는 경로입니다.
- (3) **(known after apply)의 재등장**. "새 서브넷의 id를 아직 모른다 → 그걸 참조하는 서버를 새로 만들어야 한다"는 인과가 이 한 줄에 다 들어 있습니다.

apply를 누르면 EC2가 삭제되고 퍼블릭 IP가 바뀝니다. 운영 중인 서비스였다면 장애입니다.

반드시 원래대로 되돌리기

❌ 절대 금지

`git checkout -- network.tf` / `git restore network.tf` 를 쓰지 마세요.

`practice/network.tf` 는 리포에 TODO 주석 스텔레톤 상태로 커밋돼 있습니다. git으로 되돌리면 cidr 한 글자가 아니라 A-3·A-4에서 채운 코드 전부가 사라집니다. 그러면 다음 `plan` 이 **No changes.** 가 아니라 **5 to destroy** 로 나옵니다.

직접 되돌립니다.

```
# 1) network.tf 의 cidr_block 을 "10.0.2.0/24" -> "10.0.1.0/24" 로 직접 수정
# 2) terraform.tfvars 의 instance_type 을 "t3.micro" 로 직접 수정
git diff network.tf          # 스텔레톤과의 차이 = 내가 채운 코드. cidr가 10.0.1.0/24 인지 확인
                             # (지금 cwd가 practice/ 이므로 경로에 practice/ 를 붙이지 않습니다)
terraform plan               # "No changes." 로 돌아왔는지 확인 ← 이 단계 필수
```

! 꼭 기억

No changes. 가 나오기 전에는 다음으로 넘어가지 마세요. 여기서 원복이 덜 되면 Block C의 destroy 숫자가 7이 아니게 됩니다.

- `instance_type` 변경은 ~ 인가 `-/+` 인가: `___`
- `# forces replacement` 주석이 붙은 속성 이름: `___`
- 서브넷 cidr 변경 시 숫자: `___ to add, ___ to change, ___ to destroy`
- 왜 3개인가 (참조 번호로): `___`
- apply 하지 않았음을 확인. `terraform plan` 이 `No changes.` 인가: `___`

[실습] B-7 (심화 ★★). cycle(고리) 만들어보기

지금 하는 일은 일부러 그래프에 고리를 만들어서 Terraform이 어떻게 막는지 보는 것입니다.

`compute.tf` 맨 아래에 SG 두 개를 서로 참조하도록 붙이고 `terraform plan`.

```
resource "aws_security_group" "a" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    security_groups = [aws_security_group.b.id] # a 가 b 를 기다림
  }
}
resource "aws_security_group" "b" {
  vpc_id = aws_vpc.main.id
  ingress {
    from_port      = 0
    to_port        = 0
    protocol       = "-1"
    security_groups = [aws_security_group.a.id] # b 가 a 를 기다림 ← 고리 완성
  }
}
```

Error: Cycle: aws_security_group.a, aws_security_group.b

a는 b를 기다리고 b는 a를 기다립니다. 어느 쪽도 시작할 수 없습니다.

plan 단계에서 중단되므로 AWS에는 아무 일도 일어나지 않습니다. 해결 방법은 규칙을

`aws_vpc_security_group_ingress_rule` 이라는 별도 리소스로 빼는 것입니다(개념워크북 13번).

🚫 절대 금지

확인했으면 이 블록을 반드시 지우세요.

Cycle 에러는 그래프를 만드는 단계에서 나기 때문에 `plan` 뿐 아니라 `destroy` 도 실행되지 않습니다. 지우지 않고 Block C로 가면 `destroy`가 계속 실패하고 EC2가 계정에 남아 과금됩니다.

지운 뒤 `terraform plan` 이 `No changes.` 로 실행되는 것까지 확인하고 넘어가세요.

⚠️ 주의

블록 방식 `ingress / egress` 와 별도 리소스 `aws_vpc_security_group_ingress_rule` 을 섞어 쓰지 마세요. 두 방식이 서로의 규칙을 지웁니다.

Block C. destroy + 잔존 점검 (11분)

이 단계를 하지 않으면 오늘 실습은 실패입니다

왜 이 블록이 제일 중요한가: 오늘의 비용 정리

지금 여러분 계정에는 서버 한 대가 켜져 있습니다. 켜져 있는 동안 요금이 발생합니다.

항목	시간당
EC2 <code>t3.micro</code> (서울)	\$0.0130
퍼블릭 IPv4 주소 1개	\$0.005
EBS <code>gp3</code> 8GiB	약 \$0.001
합계	약 \$0.019 (약 27원)

상황	총 비용
오늘 실습하고 바로 destroy	약 20원
destroy를 잊고 한 달 방치	약 \$13.9 (약 2만원)

차이는 `terraform destroy` 한 줄입니다.

🚫 절대 금지

"프리티어니까 괜찮겠지"로 넘기지 마세요. 프리티어가 남아 있어도 퍼블릭 IPv4는 별도로 과금될 수 있고, 2025년 이후에 만든 계정은 프리티어 체계 자체가 예전과 다릅니다. 프리티어 여부와 무관하게, destroy를 안 하면 비용이 발생할 수 있다고 생각하고 진행하세요.

[실습] C-0. 증빙 먼저 복사

! 꼭 기억

destroy하면 복구할 수 없습니다. 과제 제출의 증빙은 이 출력뿐입니다. 지금 파일로 저장하세요.

```
ID=kdh1834      # ← 여기를 본인 GitHub ID로 바꾸세요. 예시 값 그대로 두면 안 됩니다
echo "$ID"      # 바꿨는지 눈으로 확인
mkdir -p "../submissions/$ID"

terraform state list > "../submissions/$ID/state-list.txt"

# 퍼블릭 IP는 빼고, 계정번호 12자리는 <account-id> 로 가려서 저장합니다
terraform state show aws_instance.web \
  | grep -Ev 'public_ip|public_dns' \
  | sed -E 's/[0-9]{12}/<account-id>/g' >> "../submissions/$ID/state-list.txt"

jq -r '.resources[] | "\(.type).\(.name) <- \(.instances[0].dependencies // [] | join(", "))"' \
  terraform.tfstate >> "../submissions/$ID/state-list.txt"
```

💡 팁

jq 가 없다면 마지막 명령 대신 이렇게 하세요. 오늘의 핵심 증빙이라 비워두면 안 됩니다.

```
terraform show -json > /dev/null 2>&1 # jq 없이도 되는지 확인용
grep -o '"dependencies":[^\]]*' terraform.tfstate >> "../submissions/$ID/state-list.txt"
```

그래도 안 되면 B-4 ⑤에서 화면에 나온 결과를 복사해 붙여넣으셔도 됩니다.

🚫 절대 금지

커밋 전에 `state-list.txt` 를 눈으로 한 번 읽으세요. 제거해야 하는 것이 두 가지입니다. - 퍼블릭 IP. 22번 포트가 열려 있던 서버 주소입니다 - ARN 안의 계정번호 12자리. `arn:aws:ec2:ap-northeast-2:123456789012:instance/i-...`

위 `grep` 과 `sed` 가 대부분 제거하지만, 남아 있으면 `x.x.x.x / <account-id>` 로 직접 바꾸세요.

[실습] C-1. 삭제 순서를 먼저 예측하고 destroy plan 읽기

종이에 먼저 적어보세요. 의존성 지도를 뒤집으면 순서가 나옵니다.

[1단계]	_____		_____		_____
[2단계]	_____		_____		_____
[3단계]	_____				
[4단계]	_____				

이제 정답을 확인합니다.

```
terraform plan -destroy
```

```
Plan: 0 to add, 0 to change, 7 to destroy.
```

정답은 이렇습니다.

```
[1단계] aws_instance.web | aws_route_table_association.public
        (리프 노드. 아무도 이들을 참조하지 않음)
[2단계] aws_security_group.web | aws_subnet.public | aws_route_table.public
[3단계] aws_internet_gateway.gw
[4단계] aws_vpc.main                                (마지막)
```

! 꼭 기억

삭제 순서는 apply의 층 번호를 단순히 뒤집은 게 아닙니다. 의존 방향을 뒤집은 그래프에서 다시 계산합니다.

그래서 apply에서 3층이던 `instance` 와 4층이던 `association` 이 삭제에서는 같은 1단계가 됩니다. 둘 다 아무도 참조하지 않는 리프 노드라서 먼저 지울 수 있기 때문입니다.

기준

apply	"내가 무엇을 기다리나"
destroy	"나를 기다리는 게 아직 남아 있나"

그리고 그 판단의 근거가 B-4에서 본 state의 `dependencies` 배열입니다.

💡 팁

`terraform graph -type=plan-destroy` 로도 볼 수 있지만, 이 출력은 기본 graph와 달리 "[root] aws_subnet.public (expand)" 처럼 복잡합니다. 보고 싶으면 이렇게 필터링해서 사용하세요(선택).

```
terraform graph -type=plan-destroy \
| grep '\->' | grep -vE 'provider|output|root\[root\]' \
| sed 's/\[root\] //g; s/ (expand)//g; s/"//g; s/;/ /' | sort
```

[실습] C-2. destroy 실행

```
terraform destroy    # yes
```

! 꼭 기억

Plan: 0 to add, 0 to change, 7 to destroy. 줄을 먼저 읽고 `yes` 를 치세요. 숫자가 7이 아니면 리소스가 남았거나 덜 만들어진 것입니다.

⚠ 주의

EC2 삭제는 30~90초 걸립니다. Ctrl+C 금지. 서버가 완전히 `terminated` 가 될 때까지 기다려야 그 뒤에 서브넷과 SG를 지울 수 있습니다.

[확인] destroy 완료

```
aws_route_table_association.public: Destroying...
aws_instance.web: Destroying... [id=i-0123456789abcdef0]
aws_route_table_association.public: Destruction complete after 1s
aws_route_table.public: Destroying...
aws_route_table.public: Destruction complete after 1s
aws_instance.web: Still destroying... [30s elapsed]
aws_instance.web: Destruction complete after 41s
aws_security_group.web: Destroying...
aws_subnet.public: Destroying...
aws_subnet.public: Destruction complete after 1s
aws_security_group.web: Destruction complete after 1s
aws_internet_gateway.gw: Destroying...
aws_internet_gateway.gw: Destruction complete after 1s
aws_vpc.main: Destroying...
aws_vpc.main: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
```

데이터 소스 2개는 이 숫자에 안 들어갑니다. 만든 적이 없으니 지울 것도 없습니다.

[함정] destroy가 실패할 때

💡 팁

먼저 이것부터 기억하세요. `destroy` 는 몇 번을 실행해도 안전합니다. 이미 지워진 것은 건너뛰고 남은 것만 지웁니다. 실패했다고 당황해서 콘솔에서 직접 지우지 말고, 1~2분 기다렸다가 다시 실행하세요. 아래 예러의 대부분이 이 방법으로 해결됩니다.

📌 참고

왜 기다리면 되는가. EC2에는 가상 네트워크 카드가 하나 붙어 있고(정식 명칭은 ENI, 네트워크 인터페이스), 이것이 서브넷의 IP 주소를 하나 사용 중입니다. 서버가 완전히 `terminated` 가 되어야 이 네트워크 인터페이스가 해제되고, 그래야 서브넷과 SG를 지울 수 있습니다. 아래 `DependencyViolation` 예러들이 대부분 "아직 네트워크 인터페이스가 해제되지 않았다"는 뜻입니다.

예러	원인	해결
<code>DependencyViolation: The subnet 'subnet-xxx' has dependencies and cannot be deleted</code>	서버의 네트워크 카드가 아직 정리되지 않음 / 콘솔에서 직접 만든 리소스가 그 서브넷에 있음	1~2분 기다리고 <code>terraform destroy</code> 재실행. 대부분 이걸로 끝납니다

에러	원인	해결
DependencyViolation: Network vpc-xxx has some mapped public address(es)	퍼블릭 IP를 쓰는 리소스가 아직 남음	서버가 완전히 terminate될 때까지 대기 → 재실행
DependencyViolation: resource sg-xxx has a dependent object	SG가 아직 서버에 붙어 있음	동일. B-6에서 SG <code>name</code> / <code>description</code> 을 건드렸다면 먼저 원복
Error acquiring the state lock	다른 터미널에서 명령이 돌고 있거나 이전 실행이 비정상 종료됨	끝날 때까지 대기. 더 이상 사용되지 않는 잠금이면 <code>.terraform.tfstate.lock.info</code> 삭제 (Week3에서 <code>force-unlock</code> 다룹니다)
Destroy complete! Resources: 6 destroyed. (70이 아님)	중간에 실패해서 일부만 삭제됨	<code>terraform state list</code> 로 남은 것 확인 → destroy 재실행

[확인] C-3. state가 실제로 비었나: 두 가지 방법으로 확인

```
terraform state list          # 완전히 빈 출력이어야 합니다
jq '.resources | length' terraform.tfstate # 0 이어야 합니다
jq '.serial' terraform.tfstate # A-6 때보다 커져 있습니다
```

! 꼭 기억

`state list` 가 완전히 비어야 성공입니다. 데이터 소스까지 함께 사라집니다. 관리할 리소스가 없으면 값을 읽어올
이유도 없으니까요.

그런데 `terraform.tfstate` 파일 자체는 남아 있고 `serial` 은 오히려 커졌습니다. state는 남고 내용만 비었습니다.
destroy도 하나의 "저장"이니깐요.

[관찰 📖] C-2·C-3 기록

- 가장 먼저 `Destroying...` 이 뜬 리소스: / 그것이 리프 노드인 이유:
- 가장 오래 걸린 삭제와 초:
- 예측(C-1)이 틀린 지점:
- destroy 후 `state list` 줄 수:
- `resources` 배열 길이: / `serial`: → 왜 늘었을까:

[실습] C-4. 계정 잔존 점검: "state가 비었다 ≠ 계정이 비었다"

콘솔 우측 상단이 서울(ap-northeast-2)인지 먼저 확인하세요. 리전이 다르면 아무것도 안 보이는 게 당연합니다.

확인할 것	콘솔 경로	정상 상태
EC2 인스턴스	EC2 → Instances	<code>terminated</code> 만 (<code>running</code> / <code>stopped</code> 가 있으면 사고)
EBS 볼륨	EC2 → Volumes	<code>available</code> 상태인 볼륨 0개
Elastic IP	EC2 → Elastic IPs	0개
NAT Gateway	VPC → NAT Gateways	0개
네트워크 인터페이스	VPC → Network Interfaces	내 VPC의 것 없음
VPC	VPC → Your VPCs	내 VPC 없음

CLI로 한 번에 확인하는 스크립트가 리포에 있습니다.

```
bash ../scripts/check-leftover.sh # 전부 0이어야 정상
```

! 꼭 기억

CP3. 빈 `state list` 출력 + 아래 체크 결과를 Discord에 올려주세요. `[ ] EC2 terminated [ ] Volume 0 [ ] EIP 0 [ ] NAT 0 [ ] VPC 없음`

전원 CP3 확인 전에는 세션을 끝내지 않습니다.

[실습] C-5. 커밋 사고 방지

```
git status
```

파일	이래야 정상
<code>terraform.tfvars</code>	안 보임 (<code>.gitignore</code>). 내 공인 IP가 들어 있습니다
<code>terraform.tfstate</code>	안 보임
<code>state.json</code>	안 보임 ← B-4에서 만든 것
<code>*.pem</code>	안 보임 (오늘 만들지 않았지만)
<code>.terraform/</code>	안 보임
<code>practice/network.tf</code> , <code>compute.tf</code> , <code>outputs.tf</code>	보임(수정됨) → 여기서 커밋하지 말고 아래 절차대로 <code>submissions/{id}/</code> 로 복사
<code>.terraform.lock.hcl</code>	안 보임이 정상. 이미 커밋된 파일이고 내용이 안 바뀌었기 때문

🚫 절대 금지

`terraform.tfvars` / `terraform.tfstate` / `state.json` 이 목록에 보인다면 커밋하지 말고 멈추세요.
`.gitignore` 가 적용되지 않은 상태입니다. Discord에 알려주세요.

세션 후: 실습 결과 제출 (브랜치 → PR)

세션 중에는 `practice/` 에서 실습하면 됩니다. 세션이 끝나면 아래 절차를 진행하세요.

```
cd "$(git rev-parse --show-toplevel)" # 지금 cwd는 practice/ 입니다. 아래는 전부 리포 루트 기준

ID=kdh1834      # ← 여기를 본인 GitHub ID로 바꾸세요. 예시 값 그대로 두면 안 됩니다
echo "$ID"      # 바꿨는지 눈으로 확인

git switch main && git pull
git switch -c "week2/$ID"

mkdir -p "submissions/$ID"
cp practice/*.tf practice/example.tfvars "submissions/$ID/"
cp practice/.terraform.lock.hcl "submissions/$ID/"
# C-0에서 저장한 state-list.txt 는 이미 그 폴더에 있습니다 (IP·계정번호 마스킹 확인!)
# 관찰 📝 답안을 submissions/$ID/observations.md 로 정리

git add "submissions/$ID"
git status      # tfvars / tfstate / state.json 안 올라갔는지!
git commit -m "week2: $ID 실습 제출"
git push -u origin "week2/$ID"
```

복사가 끝나면 `practice/` 는 되돌려 줍니다. 다음 주에 이 리포를 다시 열 때 깔끔한 상태가 됩니다.

```
git restore practice/ # 복사를 끝낸 뒤에만 실행. 스켈레톤 상태로 되돌아갑니다
```

그다음 GitHub에서 `main` 대상 PR을 올리면 강사가 리뷰 후 머지합니다.

! 꼭 기억

제출은 `submissions/{github-id}/` 본인 폴더에 해주세요. `practice/` 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # TODO 가 사라집니다. 폴더를 나눠야 전원 PR이 다 머지됩니다.

체크포인트 (오늘 다 했는지 확인)

- [] `terraform apply` 두 번 성공 (5 added → 2 added)
- [] `state list` 에 9줄 (리소스 7 + 데이터 소스 2) 확인
- [] `state show aws_instance.web` 으로 `private_ip` · `public_ip` · `instance_state` 확인
- [] `dependencies` 배열을 열어 의존성 지도와 대조 ← 오늘의 핵심

- [] drift 만들고 `plan` 이 되돌리자고 제안하는 것 확인 (B-5)
- [] `terraform destroy` 완료 (7 destroyed) & `state list` 빈 출력
- [] 콘솔/스크립트로 EC2·Volume·EIP·NAT·VPC 잔존 0 확인 (C-4)
- [] `git status` 로 `*.tfvars` / `*.tfstate` / `state.json` 안 올라갔는지 확인

오늘의 명령 요약

의존성 지도 순서 그대로입니다.

<code>curl -4 ifconfig.me</code>	# A-2 내 공인 IP
<code>cp example.tfvars terraform.tfvars</code>	# A-2 값 채우기
	# A-3~A-4 참조 ①~⑦ 잇기
<code>terraform fmt && terraform init</code>	# A-5
<code>terraform validate</code>	# A-5 init 이후에
<code>terraform plan</code>	# A-5 "5 to add"
<code>terraform apply</code>	# A-6 네트워크 5개
<code>terraform state list</code>	# A-6 7줄
<code>terraform graph grep '\->'</code>	# A-7 그래프를 텍스트로
	# B-1 참조 ⑧~⑪ 잇기
<code>terraform plan</code>	# B-2 "2 to add"
<code>terraform apply</code>	# B-3 EC2 (30~60초, Ctrl+C 금지)
<code>terraform state show aws_instance.web</code>	# B-4 속성 확인
<code>jq 'keys' terraform.tfstate</code>	# B-4 최상위 키 7개
<code>jq -r '.resources[] .instances[0].dependencies' terraform.tfstate</code>	# B-4 ★
<code>terraform plan -refresh-only</code>	# B-5 drift만 보기
<code>terraform plan -destroy</code>	# C-1 삭제 순서 예측 채점
<code>terraform graph -type=plan-destroy</code>	# C-1 역방향 그래프
<code>terraform destroy</code>	# C-2 7개 삭제. 필수!
<code>terraform state list</code>	# C-3 빈 출력이어야 함
<code>bash ../scripts/check-leftover.sh</code>	# C-4 계정 잔존 0 확인

다음 주 예고

Week 3. 원격 Backend & State 잠금. 오늘 분석한 `terraform.tfstate` 가 그대로 문제가 됩니다.

오늘 본 것	Week3에서
state가 내 노트북에만 있다	팀원들은 이 state를 못 본다 → S3 원격 저장소
<code>serial</code> 이 저장할 때마다 +1	두 명이 동시에 apply하면 충돌 → 잠금(lock)
state가 평문 JSON	원격에 두려면 암호화·접근 제어
<code>lineage</code> 가 state의 고유 식별자 역할	잘못된 state를 덮어쓰는 사고를 어떻게 막나

예습: 원격 state가 왜 필요한지, `backend "s3"` 블록, `terraform init -migrate-state`

참고 문서

- [Create Resource Dependencies](#) : 이 실습의 기반이 된 문서
- [Manage Resource Drift](#) : B-5의 원본
- [terraform graph](#) · [terraform state](#) · [terraform show](#)
- [aws_instance](#) · [aws_subnet](#) · [aws_route_table](#) · [aws_security_group](#)
- [aws_ami \(data source\)](#) · [aws_availability_zones](#)
- [VPC 시작하기 \(AWS 한국어 문서\)](#) : AWS 네트워크가 처음이라면
- [Amazon EC2 요금](#) · [퍼블릭 IPv4 주소 요금](#)

이 문서의 plan 출력, AMI ID, AZ, 비용은 서울 리전, AWS provider 6.57.1, Terraform 1.15.8 에서 실제로 조회·검증한 값입니다. 인스턴스 ID나 IP는 예시로 바꿔 적었습니다.

! 꼭 기억

끝났으면 `terraform destroy` 를 정말로 했는지, 그리고 콘솔에 EC2가 `terminated` 인지 한 번 더 확인하세요. 삭제하지 않은 EC2는 한 달 뒤 약 2만원의 요금으로 청구됩니다.